

MBA — WEB DEVELOPMENT PROJECT REPORT

Vibe Coding with AI Agents in VS Code

TradeEasy

A Simplified Online Trading & Investment Platform

Full-Stack Build: HTML5 · CSS3 · Bootstrap 5 · JavaScript · Python (Flask) · SQLite

Developed in Visual Studio Code using an AI coding-agent (“vibe coding”) workflow

Project source: TradeEasy_Vibe_coding_with_Backend

Report generated: September 2026

Table of Contents

Click any entry to jump to that section. Each section ends with a “Back to Table of Contents” link.

- 1. Executive Summary
- 2. Project Objectives — MBA Context
- 3. Technology Stack & Tools Used
- 4. Vibe Coding Methodology — Building with an AI Agent in VS Code
- 5. System Architecture
- 6. Frontend — Pages & Features
- 7. Backend — Flask Application & API Endpoints
- 8. Database Design — SQLite Schema
- 9. Project File Structure
- 10. Setup & Run Instructions
- 11. Security Considerations & Recommended Next Steps
- 12. Roadmap — Future Improvements
- 13. Verification Performed for This Report
- 14. Learnings & Conclusion
- Appendix A — Full API Endpoint Reference
- Appendix B — Complete Project File Listing

1. Executive Summary

TradeEasy is a full-stack web application built to teach the fundamentals of investing and financial markets. Per the project's own GitHub README, it is framed first as a “Financial Literacy Web Application”: educational modules covering Stocks, Mutual Funds, Gold, and Futures & Options (F&O) sit alongside a financial-literacy self-assessment form, wrapped in the look and feel of a simplified retail trading platform, plus a user authentication system and a contact/support form.

The project pairs a static HTML5/CSS3/JavaScript frontend (styled with Bootstrap 5) with a lightweight Python Flask backend and a SQLite database for persistence. It was built as an MBA web-development submission using a “vibe coding” workflow: features were described in natural language to an AI coding agent inside VS Code, which scaffolded and iterated the HTML, Flask routes, and database code, with the student reviewing, running, and refining the output.

This report documents the project the way the combined master report documents its own projects — purpose, what was built, the tools and technology stack, the build methodology, and the files delivered — adapted here for a software/web-development deliverable rather than a financial model. It draws on two sources: the technical README shipped inside the project folder (setup instructions, security notes) and the project's GitHub-facing README (overview, skills demonstrated, roadmap) — both are reconciled throughout, and any difference between what they describe is called out where it appears (see Section 8.4).

Disclaimer (from the project README): this project was developed for educational and portfolio purposes. The information presented within the application is intended for learning and awareness and should not be considered financial advice.

[↑ Back to Table of Contents](#)

2. Project Objectives — MBA Context

As an MBA web-development exercise, the project was scoped to demonstrate the following competencies:

- Translate a business concept — a simplified, consumer-facing online trading platform — into a working, demonstrable web product.
- Practice AI-assisted (“vibe coding”) development: directing an AI coding agent in VS Code to generate and modify frontend and backend code from natural-language prompts, rather than hand-writing every line.
- Connect a static frontend to a live backend API, moving beyond flat HTML pages to a client–server model with persisted data.
- Implement core SaaS/fintech-app building blocks: user registration and login, session-based authentication, a contact/lead-capture form, and multiple product landing pages.
- Produce a working local deployment that can be run, demoed, and iterated on — with a self-documented README covering setup and known limitations, in the spirit of a real engineering handover.

2.1 Skills Demonstrated

The project's own GitHub README lists the following as the skills this build is meant to evidence — reproduced here as it frames the MBA deliverable's learning outcomes directly:

Skill Area	Skill Area	Skill Area
Python Programming	Flask Web Development	Database Management
SQLite Integration	User Authentication	HTML & CSS
Backend Development	Problem Solving	—

[↑ Back to Table of Contents](#)

3. Technology Stack & Tools Used

The stack was deliberately kept lightweight — no build tooling, bundlers, or frontend framework — so that the AI agent's generated code stayed simple to read, run, and modify inside VS Code.

Layer	Technology	Role in the Project
Markup	HTML5	8 pages: index, login, contact, stocks, fo (Futures & Options), gold, mutualfunds, literacyform
Styling	CSS3 + Bootstrap 5.3.0 (CDN) + Bootstrap Icons 1.10.0	Responsive grid, card layouts, navbar, forms, buttons
Typography	Google Fonts — Poppins (300–700 weights)	Site-wide font, loaded via CDN <link>
Client scripting	Vanilla JavaScript (Fetch API, no framework)	Form submission, session/auth state checks, async calls to the Flask API
Backend framework	Python 3 + Flask ≥ 2.0	Serves the HTML pages, exposes a JSON REST API, manages sessions
Database	SQLite (app.db, via Python's built-in sqlite3 module)	Persists users and contact-form submissions; additional tables referenced by later endpoints
Session/auth	Flask session + secrets.token_hex(16) secret key	Cookie-based session after login; guards protected API routes
Dev environment	Visual Studio Code + AI coding agent (“vibe coding”)	Code generation, iteration, and debugging assistance
Environment isolation	Python venv	Isolated install of Flask and its dependencies (Jinja2, Werkzeug, MarkupSafe, click, itsdangerous, blinker)
Version control hygiene	Git, with a project .gitignore	Excludes venv/ and the local app.db from source control

[↑ Back to Table of Contents](#)

4. Vibe Coding Methodology — Building with an AI Agent in VS Code

“Vibe coding” here means describing intent in plain language to an AI coding agent running inside VS Code, letting it generate or modify the actual HTML, Python, and SQL, and then reviewing and running the result rather than authoring every line manually. The project's own files carry clear evidence of this iterative, prompt-driven process:

4.1 Frontend-first, backend added on top

The static pages (index.html, login.html, contact.html, stocks.html, fo.html, gold.html, mutualfunds.html, literacyform.html) were built first as a standalone Bootstrap site. The backend's own README states this explicitly: “This project adds a minimal Flask backend that serves your existing HTML files (from the project root).” This is a hallmark vibe-coding pattern — asking the agent to wrap a backend around an already-working frontend, instead of designing both together from a spec.

4.2 Agent-scaffolded backend in two files

The agent generated a small, purpose-built pair of scripts rather than a full framework project: init_db.py to create the SQLite schema, and app.py to define the Flask routes and API. This matches a typical vibe-coding prompt sequence — “add a Flask + SQLite backend for these pages” — that an agent answers with the smallest working scaffold.

4.3 Iterate, then prune

A financial-literacy quiz feature was added — the frontend page literacyform.html and a matching /api/financial-literacy endpoint were built — and then rolled back: the endpoint now simply returns HTTP 410 Gone with the message “Financial literacy form storage has been removed,” and a dedicated cleanup script, drop_financial_tables.py, was generated to drop the financial_literacy_forms, financial_literacy_quiz_results, and financial_literacy tables. Building a feature, then generating a targeted script to undo just its database footprint, is characteristic of fast, conversational agent-driven iteration.

4.4 Endpoints scaffolded ahead of schema

app.py already defines full GET/POST handlers for a stock watchlist, mutual-fund investments, gold investments, and a portfolio summary (see Section 7) — each reading and writing tables that do not yet exist in init_db.py or in the current app.db (see Section 8). This is consistent with an agent being asked to “add an API for watchlists, mutual funds, gold and a portfolio view” and generating the route logic immediately, with the corresponding CREATE TABLE statements left as a follow-up step — a common, easily-missed gap in rapid AI-assisted builds.

4.5 Self-documenting output

The agent also produced the project's README.md, including a run-book (venv → install → init_db → run) and an explicit “Security & next steps” section flagging plaintext password storage and the absence of CSRF protection. Having the agent surface its own shortcuts is a practical safeguard worth keeping in any vibe-coding workflow, and is reused as the basis for Section 11 of this report.

[↑ Back to Table of Contents](#)

5. System Architecture

TradeEasy uses a simple two-tier architecture: a Flask process that both serves static frontend files and exposes a JSON API, backed by a single SQLite database file. There is no separate frontend build step and no ORM — routes talk to SQLite directly via Python's sqlite3 module.

Browser		
	GET /, /login.html, /stocks.html, ...	(static HTML/CSS/JS)
	POST/GET /api/*	(JSON, fetch())

```

v
Flask app (app.py)
|-- send_from_directory() -> serves HTML/CSS/JS files from project root
|-- session (cookie)      -> tracks logged-in username after /api/login
|-- sqlite3 connection (g._database) -> opened per-request, closed on teardown
v
SQLite database (app.db)
|-- users      (implemented)
|-- contacts   (implemented)
|-- stock_watchlist, mutual_fund_investments, gold_investments,
    portfolio_summary (queried by app.py; not yet created — see Section 8)

```

Fig. 1 — Request flow: browser → Flask (routing, sessions) → SQLite (app.db).

Static pages are served through a single catch-all route, `serve_file(filename)`, rather than Flask's `render_template/Jinja2` templating — the HTML files are returned as-is via `send_from_directory`, so all interactivity (auth state, form posting) happens client-side via JavaScript `fetch()` calls into the `/api/*` routes.

[↑ Back to Table of Contents](#)

6. Frontend — Pages & Features

Eight HTML pages make up the frontend, all sharing the same Bootstrap 5 + Bootstrap Icons + Poppins visual system loaded from CDN.

Page	Purpose	Key Sections / Elements
index.html	Landing / home page	Hero “Welcome to TradeEasy”; “Explore Investment Options” cards for Stocks, F&O, Mutual Funds, Gold; “Why Choose TradeEasy?” section
login.html	Authentication	Login and Register forms; calls <code>/api/login</code> , <code>/api/register</code> , <code>/api/logout</code> , <code>/api/check-auth</code> ; redirects based on session state
contact.html	Support / lead capture	Contact form (name, email, message) posting to <code>/api/contact</code>
stocks.html	Stocks product page	Stock investing overview and product information
fo.html	Futures & Options product page	“What are Futures?” / “What are Options?” explainers; “Popular F&O Contracts” (Nifty 50 Futures, Bank Nifty Options, Stock Futures); FAQ section
gold.html	Gold investment product page	“Gold Investment Plans” (Digital Gold, Gold Savings Plan, Physical Gold); “Why Invest in Gold?”; gold investment calculator
mutualfunds.html	Mutual funds product page	“Compare Top Performing Funds”; “Why Invest in Mutual Funds?”
literacyform.html	Financial literacy assessment	Intro screen, JavaScript-driven dynamic quiz questions, results screen (backend storage now retired — see Sections 4.3 and 7)

All forms and auth checks use the browser's `fetch()` API to call the JSON endpoints documented in Section 7, keeping the pages framework-free while still behaving like a connected application.

[↑ Back to Table of Contents](#)

7. Backend — Flask Application & API Endpoints

app.py defines a single Flask application with two static-serving routes and ten JSON API routes. Four of the API routes (marked “Session” below) require an active login and return HTTP 401 if called unauthenticated.

Route	Method	Behaviour
/	GET	Serves index.html from the project root
/ <code><path:filename></code>	GET	Generic static-file route — serves any other file in the project root (login.html, stocks.html, css/js assets, etc.)
/api/contact	POST	Saves name, email, message to the contacts table
/api/financial-literacy	POST	Retired endpoint — returns HTTP 410 Gone (“Financial literacy form storage has been removed”)
/api/watchlist	GET / POST — Session	Reads/writes the caller's stock_watchlist rows (stock_symbol, company_name, target_price, notes)
/api/mutual-funds	GET / POST — Session	Reads/writes the caller's mutual_fund_investments rows (fund_name, investment_amount, units, purchase_date, investment_type)
/api/gold-investments	GET / POST — Session	Reads/writes the caller's gold_investments rows (investment_type, quantity, purchase_price, purchase_date, notes)
/api/portfolio	GET — Session	Returns the caller's most recent row from portfolio_summary
/api/register	POST	Creates a new row in users (username, password); returns 409 if the username already exists
/api/login	POST	Validates username/password against users, sets session['username'] on success
/api/logout	POST	Clears session['username']
/api/check-auth	GET	Returns the current session's username, or 401 if not logged in

The four session-protected endpoints (/api/watchlist, /api/mutual-funds, /api/gold-investments, /api/portfolio) are fully implemented in app.py but query tables — stock_watchlist, mutual_fund_investments, gold_investments, portfolio_summary — that are not created anywhere in the current codebase (init_db.py only creates users and contacts). Calling them against a freshly initialised database will raise a sqlite3.OperationalError (“no such table”) rather than the intended 401/200 response. This is flagged as a concrete next step in Section 11.

[↑ Back to Table of Contents](#)

8. Database Design — SQLite Schema

The database is a single file, app.db, opened per-request via Flask's application context (get_db() / teardown_appcontext) and created by running init_db.py.

8.1 Tables implemented and present in app.db

Table	Columns	Used By
users	id INTEGER PK AUTOINCREMENT · username TEXT UNIQUE NOT NULL · password TEXT NOT NULL	/api/register, /api/login
contacts	id INTEGER PK AUTOINCREMENT · name, email, message TEXT NOT NULL · created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP	/api/contact

Passwords are stored as plain text in the users table — acceptable for a local MBA coursework demo, but called out explicitly in Section 11 as the top security item to fix before any real deployment.

8.2 Tables referenced in code but not yet created

app.py's watchlist, mutual-funds, gold-investments, and portfolio endpoints (Section 7) query the following tables, none of which exist in init_db.py or the shipped app.db:

- stock_watchlist — expected columns include user_id, stock_symbol, company_name, target_price, notes
- mutual_fund_investments — expected columns include user_id, fund_name, investment_amount, units, purchase_date, investment_type
- gold_investments — expected columns include user_id, investment_type, quantity, purchase_price, purchase_date, notes
- portfolio_summary — expected columns include user_id, last_updated, and summary fields returned by /api/portfolio

These are treated as planned schema extensions rather than defects in this report — see Section 11 for the recommended CREATE TABLE follow-up.

8.3 Retired tables

drop_financial_tables.py exists solely to remove three earlier tables — financial_literacy_forms, financial_literacy_quiz_results, and financial_literacy — that supported the now-retired /api/financial-literacy endpoint. Its presence is evidence of an iterate-then-prune development cycle rather than a live part of the current schema.

8.4 Documentation vs. code — a note on the financial literacy module

The project's GitHub-facing README describes the financial literacy assessment as a live feature — “Interactive financial literacy form, User response collection, Database storage” — and lists literacyform.html and app.db among the delivered files without qualification. The codebase tells a slightly different story: literacyform.html is still present and renders its quiz entirely client-side, but the backend half has been rolled back (Section 4.3) — /api/financial-literacy now returns HTTP 410 Gone, and the supporting tables no longer exist in app.db. This is a normal artifact of fast, prompt-driven iteration: the feature was built, then its storage layer was intentionally pruned, and the outward-facing README simply was not updated to match. It is noted here rather than silently corrected, since it is a realistic example of the documentation drift that vibe-coded projects need to watch for.

[↑ Back to Table of Contents](#)

9. Project File Structure

The project root (excluding the venv/ virtual-environment folder, which is git-ignored and not part of the authored codebase) contains:

File	Type	Role
app.py	Python	Flask application — routes, sessions, API logic
init_db.py	Python	Creates app.db and the users / contacts tables
drop_financial_tables.py	Python	One-off cleanup script for retired financial-literacy tables
app.db	SQLite database	Runtime data file (git-ignored)
requirements.txt	Text	Python dependency list (Flask>=2.0)
README.md	Markdown	Setup instructions and security/next-steps notes
.gitignore	Text	Excludes venv/ and app.db from version control
index.html	HTML	Landing page
login.html	HTML	Login / register page
contact.html	HTML	Contact form page
stocks.html	HTML	Stocks product page
fo.html	HTML	Futures & Options product page
gold.html	HTML	Gold investment product page
mutualfunds.html	HTML	Mutual funds product page
literacyform.html	HTML	Financial literacy assessment page

The project's GitHub README additionally names the repository tradeeasy-financial-literacy-platform and expects a /screenshots folder (Home, Login, Stocks, Mutual Funds, Gold, F&O, and Financial Literacy Form screenshots) for the project's public listing. No such folder was included in the submitted project archive, so screenshots are not reproduced in this report.

A complete file listing with line counts is provided in Appendix B.

[↑ Back to Table of Contents](#)

10. Setup & Run Instructions

Combining both project READMEs: the GitHub README's clone step and cross-platform commands, followed by the technical README's Windows-specific run-book.

Step 1 — Clone the repository

```
git clone https://github.com/yourusername/tradeeasy-financial-literacy-platform.git
cd tradeeasy-financial-literacy-platform
```

Step 2 — Create and activate a virtual environment

Windows:

```
python -m venv venv
```

```
venv\Scripts\activate
```

Linux / macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Step 3 — Install dependencies

```
pip install -r requirements.txt
```

Step 4 — Initialise the database

```
python init_db.py
```

Creates app.db with the users and contacts tables. (Run drop_financial_tables.py separately only if migrating an older database that still has the retired financial-literacy tables.)

Step 5 — Run the application

```
python app.py
```

Step 6 — Open the application

```
http://127.0.0.1:5000
```

The Flask development server runs in debug mode, serving index.html, login.html, contact.html, stocks.html, fo.html, gold.html, mutualfunds.html, and literacyform.html directly from the project root, with the API available under /api/.

[↑ Back to Table of Contents](#)

11. Security Considerations & Recommended Next Steps

The project's own README already flags several known limitations — a good practice this report carries forward, with the schema gap from Section 8.2 added as an engineering item:

- **Hash passwords** — users.password is stored in plain text; switch to werkzeug.security.generate_password_hash / check_password_hash or bcrypt before any real users touch the app.
- **Strengthen authentication** — move from the current stateless username/password check to proper session-based auth with expiry, or JWT for API clients.
- **Add input validation & CSRF protection** — form posts (/api/contact, /api/login, /api/register) currently trust client input directly.
- **Create the missing tables** — add stock_watchlist, mutual_fund_investments, gold_investments, and portfolio_summary to init_db.py so the four session-protected endpoints in Section 7 run without a sqlite3.OperationalError.
- **Move to templates** — the README itself suggests migrating the static HTML into a templates/ folder and using Flask's render_template for more advanced, server-rendered pages.
- **Add tests** — no automated tests currently cover the API; the README recommends adding simple unit tests for each route.

None of these block a local classroom demo of the working flows (registration, login/logout, contact form, and the four product pages) — they are the natural “what I’d do next” list for taking TradeEasy from a coursework prototype toward a deployable app.

[↑ Back to Table of Contents](#)

12. Roadmap — Future Improvements

The project's GitHub README lists the following as its own planned next steps. They are reproduced here as a forward-looking roadmap; several line up directly with the code-level gaps already identified in this report (Sections 7–8 and 11).

- **User Dashboard** — a consolidated, logged-in view; would sit naturally on top of the existing session/auth system (Section 7).
- **Investment Calculators** — the gold and mutual-fund pages already reference a calculator UI (Section 6); this would extend it with real computation.
- **Portfolio Tracking** — already scaffolded at the API level via `/api/watchlist`, `/api/mutual-funds`, `/api/gold-investments`, and `/api/portfolio`; the remaining work is exactly the missing tables described in Section 8.2.
- **Market Data Integration** — connecting the stocks, F&O, and gold pages to a live pricing feed instead of static content.
- **Better UI Design** — visual refinement beyond the current Bootstrap 5 default theme.
- **Admin Panel** — a management view over the users and contacts tables (and, per the README, over financial-literacy submissions, pending the documentation/code reconciliation noted in Section 8.4).

[↑ Back to Table of Contents](#)

13. Verification Performed for This Report

To ensure this report accurately reflects the delivered codebase rather than only its documentation, the following checks were made directly against the project files before writing Sections 6–11:

- Every route decorator in `app.py` was read and mapped to Section 7's API table, including which routes check session state.
- `app.db` was opened directly and its `sqlite_master` table queried, confirming only users and contacts exist — which is what surfaced the schema gap documented in Section 8.2.
- `init_db.py` and `drop_financial_tables.py` were compared against `app.py`'s table references to distinguish implemented, planned, and retired tables.
- Each HTML file's headings and `fetch()` calls were inspected to confirm the feature list in Section 6 and the auth calls in Section 7.
- `README.md` was used as the primary source for Sections 10 and 11, with its instructions reproduced rather than re-derived.

[↑ Back to Table of Contents](#)

14. Learnings & Conclusion

TradeEasy demonstrates a complete, if intentionally lightweight, full-stack build: a multi-page Bootstrap frontend, a Flask REST API, session-based authentication, and SQLite persistence, produced through an AI-assisted “vibe coding” workflow in VS Code rather than a traditional from-scratch build.

The exercise surfaces both the strength and the characteristic risk of this workflow. The strength is speed: a working login system, a contact pipeline, and four product pages with a backing API were all scaffolded and wired together quickly, with the agent even generating its own setup README and flagging its own security shortcuts. The risk, visible directly in the code, is drift between layers built in separate prompts — the watchlist, mutual-funds, gold, and portfolio endpoints were scaffolded before their tables were, so the API and the schema are not yet in sync (Sections 7–8).

As an MBA web-development deliverable, the project meets its objective: it shows a business idea — a simplified retail investment platform — taken from concept to a running, demoable full-stack application, while leaving a clear, self-documented list of next engineering steps (Section 11) for taking it further.

[↑ Back to Table of Contents](#)

A. Full API Endpoint Reference

POST /api/contact

Request body (form or JSON): name, email, message (all required).

Success: 201 { status: "ok", id }. Error: 400 if any field is missing.

POST /api/financial-literacy

Retired. Always returns 410 { error: "Financial literacy form storage has been removed" }.

GET / POST /api/watchlist (session required)

POST body: stock_symbol (required), company_name, target_price, notes. Success: 201 { status: "ok", id }.

GET: returns the caller's full watchlist as a JSON array. Both require an active session (401 otherwise).

GET / POST /api/mutual-funds (session required)

POST body: fund_name, investment_amount, units, investment_type (all required), purchase_date (optional). Success: 201.

GET: returns the caller's mutual-fund investment rows as a JSON array.

GET / POST /api/gold-investments (session required)

POST body: investment_type, quantity, purchase_price (all required), purchase_date, notes (optional). Success: 201.

GET: returns the caller's gold investment rows as a JSON array.

GET /api/portfolio (session required)

Returns the caller's most recent portfolio_summary row (404 if none exists).

POST /api/register

Body: username, password (both required). Success: 201 { status: "ok" }. 409 if the username already exists.

POST /api/login

Body: username, password. On match, sets session['username'] and returns 200 { status: "ok", user_id, username }. 401 on invalid credentials.

POST /api/logout

Clears session['username']. Always returns 200 { status: "ok" }.

GET /api/check-auth

Returns 200 { status: "ok", username } if a session is active, else 401 { status: "not_authenticated" }.

[↑ Back to Table of Contents](#)

B. Complete Project File Listing

Line counts, from the delivered project root (venv/ excluded, as it holds installed third-party packages rather than authored code):

File	Lines	Notes
index.html	746	Landing page
login.html	873	Login / register page
contact.html	838	Contact form page
stocks.html	851	Stocks product page
fo.html	728	Futures & Options product page
gold.html	881	Gold investment product page
mutualfunds.html	800	Mutual funds product page
literacyform.html	1099	Financial literacy assessment page
app.py	198	Flask application — 12 routes total
init_db.py	37	Database schema (users, contacts)
drop_financial_tables.py	25	Cleanup script for retired tables
README.md	48	Setup + security/next-steps notes
requirements.txt	1	Flask>=2.0

Total: 7,125 lines across the authored HTML, Python, and documentation files (excluding venv/).

[↑ Back to Table of Contents](#)